



Relatório de Auditoria de Segurança

ACESSO

02 de setembro de 2026

ESCOPO AUDITADO

- Backend: Site/Backend (Node.js 20+ / Express 5 / Sequelize 6 / PostgreSQL via Supabase / JWT via jsonwebtoken / Supabase Storage para arquivos).
- Frontend: Site/Frontend (React 19 / TypeScript / Vite / TanStack Router+Query / Radix UI).
- Deploy: render.yaml (Render.com) — sem Docker/CI (.github) neste repositório.
- 23 arquivos de rota do backend, todos os controllers/services associados, e os componentes de frontend com renderização de conteúdo de usuário ou controle de acesso por papel.

NOTA METODOLÓGICA

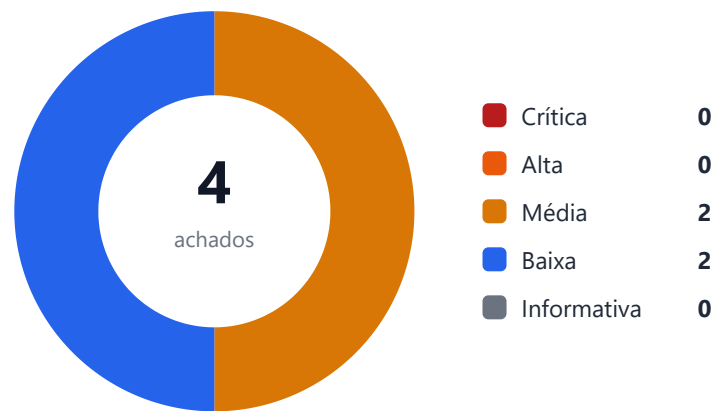
Cada categoria pedida foi mapeada para o equivalente real desta stack (Node.js/Express/Sequelize/PostgreSQL/JWT no backend, React/TypeScript no frontend, deploy via Render sem Docker/CI) antes da verificação:

1. Banco sem tranca (isolamento)	Este projeto NÃO usa Supabase RLS como mecanismo de isolamento: o backend conecta ao Postgres diretamente via Sequelize com credenciais completas (config/database.js), o que bypassa qualquer RLS que a UI do Supabase possa ter. A autoridade real de isolamento é o filtro manual por `req.user.id`/posse em cada service, mais um mecanismo de bloqueio social ('BloqueioService') que várias telas precisam respeitar. A auditoria verificou a presença desse filtro em TODOS os handlers, e a aplicação consistente do bloqueio nos pontos de listagem social.
2. Permissão definida no navegador	Mapeado para `rbacMiddleware(...papeis)` (Express) cruzado com os gates de UI em React ('GuardaAcesso', condicionais por `user.tipo`). Verificado se cada rota que o frontend esconde por papel tem o rbac correspondente no backend.
3. IDOR	Percorridos sistematicamente os 23 arquivos de rota do backend (não amostra) e os métodos de serviço correspondentes, verificando se todo handler que recebe um ID (path/query/body) confirma posse ('garantirDono', 'garantirParticipante', 'garantirConversa', 'buscarProprio', ou filtro direto no WHERE) antes de ler/alterar/remover.
4. Chaves expostas	Verificado config/env.js (fail-fast de variáveis obrigatórias), .env.example, histórico completo do `git log -p`, e o bundle de produção do frontend (`npm run build` + grep no `dist/`).
5. Inputs sem tratamento (XSS)	Sem framework de templates server-side para HTML de resposta (API é JSON puro) — a superfície real é: (a) e-mails HTML gerados no backend (utils/emailTemplates.js) e (b) renderização de conteúdo de usuário no frontend React. Verificada a função de escape usada nos e-mails e todo uso de `dangerouslySetInnerHTML`/'innerHTML`/href` com dado controlado pelo usuário no frontend.

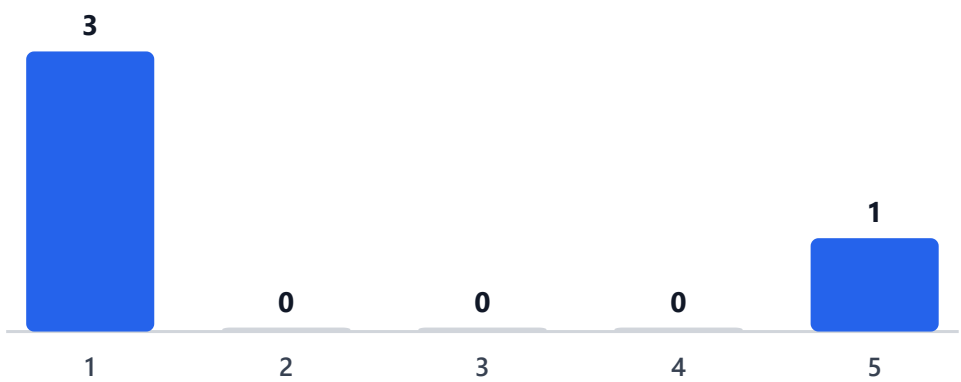
Resumo executivo

Foram identificados **4 achados** nas cinco categorias solicitadas, todos verificados diretamente no código-fonte e, quando aplicável, confirmados por teste ao vivo via API antes de entrarem neste relatório. Não foram encontrados achados de severidade **crítica** ou **alta**. As categorias 2 (permissão definida só no navegador), 3 (IDOR) e 4 (chaves expostas) foram auditadas de forma exaustiva sem gerar nenhum achado adicional além dos listados — a extensa lista de "pontos fortes" a seguir documenta essa cobertura.

Achados por severidade



Achados por categoria



1 — 1. Banco sem tranca 2 — 2. Permissão no navegador 3 — 3. IDOR 4 — 4. Chaves expostas
5 — 5. Inputs sem tratamento (XSS)

Severidade	Quantidade
Crítica	0
Alta	0
Média	2
Baixa	2
Informativa	0

Pontos fortes verificados

Itens confirmados como corretamente implementados durante a auditoria — listados com evidência específica para demonstrar a abrangência real da verificação, não apenas a ausência de achados.

✓ IDOR — verificação de posse consistente em toda a base

23/23 arquivos de rota auditados. Padrões centralizados e reaproveitados: ``garantirDono()`` (utils/authorization.js:51-61, compara ``String(donoid) !== String(usuario.id)``, admin sempre passa), ``garantirParticipante()`` (ConversaService.js:80-86), ``garantirConversa()`` (ChatbotService.js:353-374), ``buscarProprio()`` (PerfilCandidatoService.js:190-208). Notificações e favoritos vão além: filtram ``usuarioid`` DIRETO no WHERE da query (NotificacaoService.js:187-189, 221-223), tornando o objeto de outro usuário literalmente inexistente para a consulta — nunca revela sequer que o ID existe.

✓ RBAC — nenhuma rota administrativa depende só do frontend

``adminRoutes.js:17`` aplica ``router.use(authMiddleware, rbacMiddleware("administrador"))`` uma única vez para TODAS as ~30 rotas do painel — proteção de bloco, não rota a rota. Frontend replica isso via layout ``admin.tsx`` (``GuardaAcesso`` tipos = ``{"administrador"}``), mas a segurança real está no middleware do backend, confirmado independentemente. Rotas empresa-only (``vagaRoutes.js``) e candidato-only (``dashboardRoutes.js``, ``perfilRoutes.js``) seguem o mesmo padrão.

✓ Ações administrativas sensíveis têm proteção extra além do papel

``UsuarioService.setAtivo()`` (linha 172-182) usa ``garantirAlvoDeAcaoAdministrativa`` para impedir um admin de desativar a própria conta OU outra conta administrativa — proteção contra auto-lockout e abuso admin-sobre-admin, além do simples ``rbacMiddleware``.

✓ Upload de arquivo — defesa em profundidade completa

``uploadMiddleware.js``: nome de arquivo sempre gerado no servidor (``crypto.randomUUID()``, nunca o nome original do cliente); extensão validada por allowlist; assinatura binária (magic bytes) conferida contra o mimetype declarado (``assinaturaValida()``, linhas 80-143) — um PDF renomeado para ``.mp4`` é rejeitado mesmo passando pela checagem de ``Content-Type``; limite de tamanho por tipo de arquivo (vídeo nunca reaproveita o limite de imagem).

✓ Segredos — nenhum hardcode, nenhum default perigoso

``config/env.js``: ``JWT_SECRET``/credenciais de banco são OBRIGATÓRIOS (``process.exit(1)`` se ausentes) — sem fallback para um valor fixo; ``JWT_SECRET`` exige ≥32 caracteres. Chaves opcionais (``SUPABASE_SERVICE_ROLE_KEY``, ``OPENROUTER_API_KEY``, ``BREVO_API_KEY``) caem em ``null`` (funcionalidade desabilitada), nunca em uma string-secredo padrão. ``.env.example`` contém apenas placeholders. ``git log -p`` completo sem nenhum segredo commitado. Bundle de produção do frontend (``npm run build`` + ``grep`` no ``dist/``) sem nenhuma chave de servidor embutida — só ``VITE_API_URL`` (uma URL pública) é exposta ao cliente.

✓ XSS — nenhuma renderização de HTML de usuário sem escape

Conteúdo de usuário (postagens, comentários, mensagens) sempre renderizado via interpolação JSX (``{variavel}``), nunca ``dangerouslySetInnerHTML`` — React escapa automaticamente. Os 2 únicos usos de ``dangerouslySetInnerHTML`/`innerHTML`` no frontend inteiro (VLibras.tsx:36 e ui/chart.tsx:73) usam string estática ou configuração fixa do desenvolvedor, nunca dado de usuário — verificado nos dois casos. Único campo de URL controlado pelo usuário renderizado como ``href`` (``empresa.site``) é validado com ``isURL()`` (allowlist de protocolo http/https/ftp) tanto na criação (``authValidator.js``) quanto na edição (``empresaValidator.js``) — testado ao vivo que ``javascript:`` e ``data:`` são rejeitados por essa validação.

✓ E-mails HTML — escape consistente em 100% dos templates

``emailTemplates.js``: toda variável potencialmente controlada por usuário (``nome``, ``motivo``) passa por ``escaparHtml()`` (escapa ``&`,`<`,`>`,`"`,`'``) antes de entrar no HTML — inclusive dentro do ``layoutBase()`` compartilhado, que escapa a saudação inteira como camada extra.

✓ SQL Injection — Sequelize parametrizado; os 2 usos de SQL bruto são seguros

Todas as queries usam o query builder do Sequelize (prepared statements). Os únicos 2 usos de `sequelize.literal()` no projeto foram inspecionados: `CompartilhamentoService.js:94` usa uma string fixa sem interpolação de variável; `DenunciaService.js:537-539` interpola um valor via `sequelize.escape()`, a forma correta de sanitizar ao construir SQL bruto.

✓ Rate limiting e proteção contra força bruta

`authLimiter` (10 tentativas/15min, `skipSuccessfulRequests`) aplicado a 14 rotas de autenticação (`authRoutes.js`) — login, cadastro, recuperação de senha, 2FA. `apiLimiter` (300/15min) global em `/api`. Limiters dedicados por finalidade para denúncias e sugestão de IA, chaveados por usuário autenticado, não só por IP.

✓ Erros — stack trace nunca vaza em produção

`errorMiddleware.js`: `...(env.isProducao ? {} : { stack: err.stack })` — o campo `stack` só é incluído na resposta HTTP quando `NODE_ENV !== "production"`. Logs de erro detalhados ficam só no `console.error` do servidor.

✓ CORS restrito por allowlist explícita

`app.js:26-38`: função de origem customizada que só libera origens em `env.security.corsOrigins` (derivado de `FRONTEND_URL`) — nunca um wildcard `*`, mesmo com `credentials: true`.

✓ Fallback de código por e-mail (dev) nunca vaza em produção

Tanto `RecuperacaoSenhaService.js` quanto `authService.js` (troca de e-mail) só fazem `console.info` do código de verificação quando `EmailService` está indisponível E `process.env.NODE_ENV !== "production"` — o fallback de desenvolvimento é explicitamente desligado em produção.

Pontos fracos — riscos centrais

Síntese dos riscos reais identificados, detalhados achado a achado na seção seguinte.

- **Isolamento social incompleto num único ponto de listagem** — a aba de compartilhamentos do perfil não aplica a mesma checagem de bloqueio que todo o resto do sistema já aplica corretamente (achado A1).
- **Caminho legado ainda ativo para seguir empresa** — um endpoint já marcado como depreciado no próprio código continua contornando bloqueio (achado A2).
- **Superfície de upload sem uso funcional** — duas rotas genéricas de upload, sem nenhum consumidor no frontend, aceitam arquivos sem vínculo a um recurso do usuário (achado A3).
- **Tokens de sessão em localStorage** — gap de defesa em profundidade caso um XSS seja introduzido no futuro; não há XSS ativo hoje (achado A4).

Achados detalhados por categoria

1. Banco sem tranca

Severidade	Arquivo:linha	Descrição
Média	Site/Backend/src/services/CompartilhamentoService.js linhas 79-103	Bloqueio entre usuários não é aplicado em `GET /compartilhamentos/usuario/:usuariold`
Média	Site/Backend/src/services/InteracaoService.js linhas 100-120 (rota: Site/Backend/src/routes/empresaRoutes.js, 53-60)	Endpoint duplicado `POST /empresas/:id/seguir` ignora bloqueio (bypass confirmado)
Baixa	Site/Backend/src/routes/uploadRoutes.js linhas 24-29, 38-43	Rotas de upload genéricas sem associação a um recurso do usuário

Média

A1 Bloqueio entre usuários não é aplicado em `GET /compartilhamentos/usuario/:usuariold`

Arquivo: Site/Backend/src/services/CompartilhamentoService.js — linhas 79-103

```
async listarPorUsuario(usuarioId, query, solicitante) {  
  ...  
  if (solicitante && !ehAdministrador(solicitante)) {  
    const idsSeguidos = await SeguidorService.idsSeguidos(solicitante.id);  
    wherePostagem[Op.or] = [  
      sequelize.literal(  
        `EXISTS (SELECT 1 FROM usuarios u WHERE u.id = "postagem"."usuario_id"  
          AND (u.perfil_publico = true OR u.tipo_usuario = 'empresa'))`  
      ),  
      { usuarioId: { [Op.in]: [...idsSeguidos, solicitante.id] } }  
    ];  
    // <- nunca consulta BloqueioService aqui  
  }  
}
```

Descrição: O método que alimenta a aba "Compartilhamentos" do perfil filtra por perfil público/seguidor, mas nunca chama `BloqueioService`/`garantirAcessoAPostagem` — ao contrário de TODO outro ponto de acesso a conteúdo social do projeto (feed geral, busca, comentários, curtidas e até o método irmão `listarPorPostagem` da mesma classe, linha 49-50, que corretamente reaproveita `garantirAcessoAPostagem`).

Por que é explorável: Um usuário bloqueado (em qualquer direção) continua enxergando a lista de publicações que a outra parte compartilhou, desde que a publicação original seja pública — violando a regra de isolamento social que o próprio projeto define e aplica em todos os outros lugares (Fase 9, Bloco 2).

Evidência: Teste ao vivo (script descartável, contas e2e-audit-*): A bloqueia B; B compartilha uma publicação pública; GET /compartilhamentos/usuario/:idDeB feito por A retorna HTTP 200 com 1 item — deveria devolver lista vazia ou 403, como o feed geral corretamente faz para o mesmo par bloqueado (GET /postagens?usuariold=... retornou 403 no mesmo teste).

Condições de exploração: Não depende de nenhuma flag ou configuração — reproduzível em qualquer ambiente sempre que existir um compartilhamento de uma publicação pública entre as duas partes.

Média A2 Endpoint duplicado `POST /empresas/:id/seguir` ignora bloqueio (bypass confirmado)

Arquivo: Site/Backend/src/services/InteracaoService.js — linhas 100–120 (rota: Site/Backend/src/routes/empresaRoutes.js, 53–60)

```
async alternarSeguir(empresaId, solicitante) {
  const candidato = await this.candidatoDoUsuario(solicitante.id);
  const empresa = await Empresa.findByPk(empresaId);
  if (!empresa) throw ApiError.notFound("Empresa não encontrada.");
  const existente = await EmpresaSeguida.findOne({
    where: { candidatoId: candidato.id, empresaId }
  });
  // <- nunca chama BloqueioService.estaBloqueadoEntre aqui
  if (existente) { await existente.destroy(); return { seguindo: false }; }
  await EmpresaSeguida.create({ candidatoId: candidato.id, empresaId });
  return { seguindo: true };
}
```

Descrição: Existe um segundo caminho para seguir empresa, gravando na mesma tabela `EmpresaSeguida` que `SeguidorService.alternarEmpresa` (o caminho "oficial", usado pelo frontend), mas sem nenhuma checagem de bloqueio. Já está marcado como `@deprecated` no próprio código-fonte (Fase 9, Bloco 5) e nenhum consumidor foi encontrado no frontend/App/scripts — mas a rota continua montada e acessível a qualquer candidato autenticado.

Por que é explorável: Um candidato bloqueado por uma empresa (ou que bloqueou a empresa) consegue segui-la mesmo assim, chamando `POST /empresas/:id/seguir` em vez de `POST /seguir/empresas/:id` — o endpoint correto recusa com 403, este aceita com 200.

Evidência: Teste ao vivo: candidato bloqueia o usuário da empresa; POST /seguir/empresas/:id (endpoint novo) → 403, como esperado; POST /empresas/:id/seguir (endpoint antigo, mesmo candidato, mesma empresa, bloqueio ainda ativo) → 200, seguimento criado.

Condições de exploração: Requer conhecer o ID da empresa (público) — nenhuma outra pré-condição. Já documentado como risco conhecido no código-fonte; esta auditoria reconfirma que o bypass continua ativo hoje.

Baixa A3 Rotas de upload genéricas sem associação a um recurso do usuário

Arquivo: Site/Backend/src/routes/uploadRoutes.js — linhas 24–29, 38–43

```
router.post("/imagem", uploadImagem.single("arquivo"),
  processarArmazenamento, UploadController.imagem);
...
router.post("/anexos", uploadAnexos.array("arquivos", 4),
  processarArmazenamento, UploadController.anexos);
// processarArmazenamento = criarProcessadorArmazenamento({}) -> pasta vazia,
// sempre bucket público, sem prefixo por usuário
```

Descrição: Diferente das demais rotas de upload do projeto (foto/capa de perfil, anexo de postagem, currículo — todas gravam em uma pasta com o `usuariold` do dono, ex. `postagens/<usuariold>/...`), estas duas rotas usam `criarProcessadorArmazenamento({})` sem `pasta`, gravando direto na raiz do bucket público, sem nenhuma amarração ao usuário ou a um registro no banco.

Por que é explorável: Qualquer usuário autenticado (candidato, empresa ou administrador) pode fazer upload de imagens arbitrárias para a raiz do bucket público repetidamente, sem cota por usuário nem rastro de dono — o único limite é o `apiLimiter` genérico (300 req/15min por IP). Confirmado que nenhum consumidor do frontend chama estas duas rotas (`grep` no Site/Frontend não encontrou nenhuma chamada a `/upload/imagem` ou `/upload/anexos`) — é superfície de ataque exposta sem uso funcional real.

Evidência: Leitura de código: `uploadMiddleware.js` linha ~305 define `processarArmazenamento = criarProcessadorArmazenamento({})`; comparado com `postagemRoutes.js`, que sempre passa `pasta: (req) => `postagens/\${req.user.id}` (escopado). Busca em Site/Frontend/src por `upload/imagem` e `upload/anexos`: nenhuma ocorrência.

Condições de exploração: Requer apenas uma conta autenticada (qualquer papel) — nenhuma configuração especial.

2. Permissão no navegador

Severidade	Arquivo:linha	Descrição
<i>Nenhum achado adicional nesta categoria além do que já consta nos pontos fortes.</i>		

3. IDOR

Severidade	Arquivo:linha	Descrição
<i>Nenhum achado adicional nesta categoria além do que já consta nos pontos fortes.</i>		

4. Chaves expostas

Severidade	Arquivo:linha	Descrição
<i>Nenhum achado adicional nesta categoria além do que já consta nos pontos fortes.</i>		

5. Inputs sem tratamento (XSS)

Severidade	Arquivo:linha	Descrição
Baixa	Site/Frontend/src/services/api.ts linhas 8-9, 21, 26, 31, 35-36	Tokens de sessão (JWT de acesso e de renovação) guardados em localStorage

Baixa

A4 Tokens de sessão (JWT de acesso e de renovação) guardados em localStorage

Arquivo: Site/Frontend/src/services/api.ts — linhas 8-9, 21, 26, 31, 35-36

```
export const ACCESS_TOKEN_KEY = "acesso:accessToken";
export const REFRESH_TOKEN_KEY = "acesso:refreshToken";
...
export function getAccessToken(): string | null {
  return window.localStorage.getItem(ACCESS_TOKEN_KEY);
}
...
if (accessToken) window.localStorage.setItem(ACCESS_TOKEN_KEY, accessToken);
if (refreshToken) window.localStorage.setItem(REFRESH_TOKEN_KEY, refreshToken);
```

Descrição: Tanto o token de acesso quanto o de renovação (refresh) ficam em `localStorage`, acessível a qualquer script que rode no mesmo domínio. Esta auditoria não encontrou nenhum XSS ativo hoje (item 5 do escopo, verificado exaustivamente: nenhum `dangerouslySetInnerHTML`/`innerHTML` com dado de usuário, e-mails HTML sempre escapados, único `href` com URL de usuário validado por allowlist de protocolo) — mas caso um XSS seja introduzido no futuro (nova dependência, novo componente com renderização rica), o impacto passa de "execução de script" para "sequestro completo de sessão", porque um `httpOnly` cookie não teria esse problema (JavaScript não consegue lê-lo).

Por que é explorável: Não há exploração direta hoje — é um item de defesa em profundidade (o "segundo degrau" de proteção que falta caso o "primeiro degrau", ausência de XSS, seja rompido no futuro).

Evidência: Leitura de código confirmando o mecanismo de armazenamento; nenhum XSS explorável foi encontrado para efetivamente extrair esses valores no estado atual do código.

Condições de exploração: Só se torna criticamente explorável SE E QUANDO um XSS futuro for introduzido em qualquer página autenticada — condição hipotética, não presente hoje.

Recomendações priorizadas

P1 — Corrigir no próximo ciclo (bypass de isolamento social já ativo)

1. **[A1]** Fazer CompartilhamentoService.listarPorUsuario reaproveitar garantirAcessoAPostagem/BloqueioService, igual a todo outro ponto de listagem social do sistema.
2. **[A2]** Remover (ou, no mínimo, aplicar BloqueioService.estaBloqueadoEntre em) o endpoint depreciado POST /empresas/:id/seguir, já que não há nenhum consumidor no frontend hoje.

P2 — Reduzir superfície de ataque desnecessária

1. **[A3]** Remover as rotas POST /upload/imagem e POST /upload/anexos (sem consumidor) ou, se forem necessárias para uso futuro, escopá-las por usuário como toda outra rota de upload do projeto já faz.

P3 — Defesa em profundidade (sem urgência, sem achado ativo hoje)

1. **[A4]** Avaliar migrar o armazenamento dos tokens JWT de localStorage para cookies httpOnly+Secure+SameSite, como proteção adicional caso um XSS seja introduzido futuramente. Mudança arquitetural maior — não é uma correção pontual.

Issues para o GitHub

Texto pronto para copiar e colar na criação de cada issue no GitHub, um bloco por achado, delimitado por --- ISSUE n --- / --- FIM ISSUE n ---. Nenhum achado foi agrupado nesta rodada: os quatro tratam de arquivos, causas-raiz e correções suficientemente distintas para justificar issues separadas.

```

--- ISSUE 1 ---
### Título
[Segurança] Bloqueio entre usuários não é aplicado em `GET /compartilhamentos/usuario/:usuarioId`

### Labels sugeridas
security, severidade:media

### Severidade
Média

### Descrição do problema
O método que alimenta a aba "Compartilhamentos" do perfil filtra por perfil público/seguidor, mas nunca chama `BloqueioService`/`garantirAcessoAPostagem` – ao contrário de TODO outro ponto de acesso a conteúdo social do projeto (feed geral, busca, comentários, curtidas e até o método irmão `listarPorPostagem` da mesma classe, linha 49-50, que corretamente reaproveita `garantirAcessoAPostagem`).

### Por que é explorável
Um usuário bloqueado (em qualquer direção) continua enxergando a lista de publicações que a outra parte compartilhou, desde que a publicação original seja pública – violando a regra de isolamento social que o próprio projeto define e aplica em todos os outros lugares (Fase 9, Bloco 2).

### Evidência
Arquivo: `Site/Backend/src/services/CompartilhamentoService.js` – linhas 79-103

```js
async listarPorUsuario(usuarioId, query, solicitante) {
 ...
 if (solicitante && !ehAdministrador(solicitante)) {
 const idsSeguidos = await SeguidorService.getIdsSeguidos(solicitante.id);
 wherePostagem[Op.or] = [
 sequelize.literal(
 `EXISTS (SELECT 1 FROM usuarios u WHERE u.id = "postagem"."usuario_id"
 AND (u.perfil_publico = true OR u.tipo_usuario = 'empresa'))`
),
 { usuarioId: { [Op.in]: [...idsSeguidos, solicitante.id] } }
];
 // <- nunca consulta BloqueioService aqui
 }
 ...
}
```

Teste ao vivo (script descartável, contas e2e-audit-*): A bloqueia B; B compartilha uma publicação pública; GET /compartilhamentos/usuario/:idDeB feito por A retorna HTTP 200 com 1 item – deveria devolver lista vazia ou 403, como o feed geral corretamente faz para o mesmo par bloqueado (GET /postagens?usuarioId=... retornou 403 no mesmo teste).

### Condições de exploração
Não depende de nenhuma flag ou configuração – reproduzível em qualquer ambiente sempre que existir um compartilhamento de uma publicação pública entre as duas partes.

### Correção sugerida
Substituir a montagem manual de `wherePostagem[Op.or]` em `listarPorUsuario` por uma chamada a `garantirAcessoAPostagem` (ou ao mesmo filtro SQL que `PostagemService.findAll` já usa) antes de listar os compartilhamentos, garantindo que um bloqueio em qualquer direção resulte em lista vazia/403, igual ao restante do sistema.

### Critérios de aceite
- [ ] Correção aplicada no arquivo/linhas indicados acima
- [ ] Teste automatizado ou script de verificação cobrindo o cenário de exploração descrito
- [ ] Regressão manual das funcionalidades relacionadas (listagem/seguir/upload conforme o caso)
- [ ] Nenhuma nova rota/consulta perde a checagem equivalente em outros pontos do sistema
--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```

```

### Título
[Segurança] Endpoint duplicado `POST /empresas/:id/seguir` ignora bloqueio (bypass confirmado)

### Labels sugeridas
security, severidade:media

### Severidade
Média

### Descrição do problema
Existe um segundo caminho para seguir empresa, gravando na mesma tabela `EmpresaSeguida` que `SeguidorService.alternarEmpresa` (o caminho "oficial", usado pelo frontend), mas sem nenhuma checagem de bloqueio. Já está marcado como `@deprecated` no próprio código-fonte (Fase 9, Bloco 5) e nenhum consumidor foi encontrado no frontend/App/scripts – mas a rota continua montada e acessível a qualquer candidato autenticado.

### Por que é explorável
Um candidato bloqueado por uma empresa (ou que bloqueou a empresa) consegue segui-la mesmo assim, chamando `POST /empresas/:id/seguir` em vez de `POST /seguir/empresas/:id` – o endpoint correto recusa com 403, este aceita com 200.

### Evidência
Arquivo: `Site/Backend/src/services/InteracaoService.js` – linhas 100-120 (rota: Site/Backend/src/routes/empresaRoutes.js, 53-60)

```js
async alternarSeguir(empresaId, solicitante) {
 const candidato = await this.candidatoDoUsuario(solicitante.id);
 const empresa = await Empresa.findByPk(empresaId);
 if (!empresa) throw ApiError.notFound("Empresa não encontrada.");
 const existente = await EmpresaSeguida.findOne({
 where: { candidatoId: candidato.id, empresaId }
 });
 // <- nunca chama BloqueioService.estaBloqueadoEntre aqui
 if (existente) { await existente.destroy(); return { seguindo: false }; }
 await EmpresaSeguida.create({ candidatoId: candidato.id, empresaId });
 return { seguindo: true };
}
```

Teste ao vivo: candidato bloqueia o usuário da empresa; POST /seguir/empresas/:id (endpoint novo) → 403, como esperado; POST /empresas/:id/seguir (endpoint antigo, mesmo candidato, mesma empresa, bloqueio ainda ativo) → 200, seguimento criado.

### Condições de exploração
Requer conhecer o ID da empresa (público) – nenhuma outra pré-condição. Já documentado como risco conhecido no código-fonte; esta auditoria reconfirma que o bypass continua ativo hoje.

### Correção sugerida
Remover a rota `POST /empresas/:id/seguir` e o método `InteracaoService.alternarSeguir` (já marcados como `@deprecated`, sem consumidor no frontend). Se por algum motivo precisar ser mantido temporariamente, aplicar `BloqueioService.estaBloqueadoEntre` antes de criar o registro em `EmpresaSeguida`, replicando a checagem que `SeguidorService.alternarEmpresa` já faz.

### Critérios de aceite
- [ ] Correção aplicada no arquivo/linhas indicados acima
- [ ] Teste automatizado ou script de verificação cobrindo o cenário de exploração descrito
- [ ] Regressão manual das funcionalidades relacionadas (listagem/seguir/upload conforme o caso)
- [ ] Nenhuma nova rota/consulta perde a checagem equivalente em outros pontos do sistema
--- FIM ISSUE 2 ---

--- ISSUE 3 ---

### Título
[Segurança] Rotas de upload genéricas sem associação a um recurso do usuário

### Labels sugeridas
security, severidade:baixa

### Severidade
Baixa

### Descrição do problema
Diferente das demais rotas de upload do projeto (foto/capa de perfil, anexo de postagem, currículo – todas gravam em uma pasta com o `usuarioId` do dono, ex. `postagens/<usuarioId>/...`), estas duas rotas usam `criarProcessadorArmazenamento({})` sem

```

`pasta`, gravando direto na raiz do bucket público, sem nenhuma amarração ao usuário ou a um registro no banco.

Por que é explorável

Qualquer usuário autenticado (candidato, empresa ou administrador) pode fazer upload de imagens arbitrárias para a raiz do bucket público repetidamente, sem cota por usuário nem rastro de dono – o único limite é o `apilimiter` genérico (300 req/15min por IP). Confirmado que nenhum consumidor do frontend chama estas duas rotas (`grep` no Site/Frontend não encontrou nenhuma chamada a `/upload/imagem` ou `/upload/anexos`) – é superfície de ataque exposta sem uso funcional real.

Evidência

Arquivo: `Site/Backend/src/routes/uploadRoutes.js` – linhas 24-29, 38-43

```
```js
router.post("/imagem", uploadImagem.single("arquivo"),
 processarArmazenamento, UploadController.imagem);
...
router.post("/anexos", uploadAnexos.array("arquivos", 4),
 processarArmazenamento, UploadController.anexos);
// processarArmazenamento = criarProcessadorArmazenamento({}) -> pasta vazia,
// sempre bucket público, sem prefixo por usuário
```
```

Leitura de código: `uploadMiddleware.js` linha ~305 define `processarArmazenamento = criarProcessadorArmazenamento({})`; comparado com `postagemRoutes.js`, que sempre passa `pasta: (req) => `postagens/\${req.user.id}` (escopado). Busca em Site/Frontend/src por `/upload/imagem` e `/upload/anexos`: nenhuma ocorrência.

Condições de exploração

Requer apenas uma conta autenticada (qualquer papel) – nenhuma configuração especial.

Correção sugerida

Remover `POST /upload/imagem` e `POST /upload/anexos` de `uploadRoutes.js` caso não haja plano de uso; caso contrário, trocar `processarArmazenamento` por um processador com `pasta: (req) => `uploads/\${req.user.id}` (mesmo padrão de `processarDocumentoPrivado`, já existente no mesmo arquivo).

Critérios de aceite

- [] Correção aplicada no arquivo/linhas indicados acima
 - [] Teste automatizado ou script de verificação cobrindo o cenário de exploração descrito
 - [] Regressão manual das funcionalidades relacionadas (listagem/seguir/upload conforme o caso)
 - [] Nenhuma nova rota/consulta perde a checagem equivalente em outros pontos do sistema
- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título

[Segurança] Tokens de sessão (JWT de acesso e de renovação) guardados em localStorage

Labels sugeridas

security, severidade:baixa

Severidade

Baixa

Descrição do problema

Tanto o token de acesso quanto o de renovação (refresh) ficam em `localStorage`, acessível a qualquer script que rode no mesmo domínio. Esta auditoria não encontrou nenhum XSS ativo hoje (item 5 do escopo, verificado exaustivamente: nenhum `dangerouslySetInnerHTML`/`innerHTML` com dado de usuário, e-mails HTML sempre escapados, único `href` com URL de usuário validado por allowlist de protocolo) – mas caso um XSS seja introduzido no futuro (nova dependência, novo componente com renderização rica), o impacto passa de "execução de script" para "sequestro completo de sessão", porque um `httpOnly` cookie não teria esse problema (JavaScript não consegue lê-lo).

Por que é explorável

Não há exploração direta hoje – é um item de defesa em profundidade (o "segundo degrau" de proteção que falta caso o "primeiro degrau", ausência de XSS, seja rompido no futuro).

Evidência

Arquivo: `Site/Frontend/src/services/api.ts` – linhas 8-9, 21, 26, 31, 35-36

```
```js
export const ACCESS_TOKEN_KEY = "acesso:accessToken";
export const REFRESH_TOKEN_KEY = "acesso:refreshToken";
...
export function getAccessToken(): string | null {
 return window.localStorage.getItem(ACCESS_TOKEN_KEY);
}
```

```
}
...
if (accessToken) window.localStorage.setItem(ACCESS_TOKEN_KEY, accessToken);
if (refreshToken) window.localStorage.setItem(REFRESH_TOKEN_KEY, refreshToken);
...
```

Leitura de código confirmando o mecanismo de armazenamento; nenhum XSS explorável foi encontrado para efetivamente extrair esses valores no estado atual do código.

#### ### Condições de exploração

Só se torna criticamente explorável SE E QUANDO um XSS futuro for introduzido em qualquer página autenticada – condição hipotética, não presente hoje.

#### ### Correção sugerida

Planejar (em uma fase própria, com testes de regressão de toda a autenticação) a migração dos tokens de acesso/renovação para cookies `httpOnly; Secure; SameSite=Strict`, ajustando `api.ts` e o middleware de autenticação do backend para ler o token do cookie em vez do header `Authorization`.

#### ### Critérios de aceite

- [ ] Correção aplicada no arquivo/linhas indicados acima
  - [ ] Teste automatizado ou script de verificação cobrindo o cenário de exploração descrito
  - [ ] Regressão manual das funcionalidades relacionadas (listagem/seguir/upload conforme o caso)
  - [ ] Nenhuma nova rota/consulta perde a checagem equivalente em outros pontos do sistema
- FIM ISSUE 4 ---